What Does "Now" Mean to Your Computer-Use Agent?
Stale Screenshots, Page Loads, and Which Check to Run Before a Click

Vasundra Srinivasan

Principal AI Engineer, Author—*Data Engineering for Multimodal AI* (O'Reilly), Stanford School of Engineering · October 2026

Abstract

Computer-use agents plan on a screenshot for several seconds and then click at a position on the screen; if the page has changed in the meantime, the click lands on an unintended element. Prior work treats the planning interval as the source of this risk. Measuring planned clicks on 31 public web pages, we find that the risk lies in page loading: 0.4% of clicks planned on fully loaded pages became stale within 30 seconds, against 17% of clicks planned in the first second after a page opened. Waiting about one second for network activity to stop removes most of this risk, yet the wait rules of eight of ten widely used open-source agents leave more than 1% of clicks stale. Where a check before the click is still needed, a pixel comparison verifies the target's position as accurately as a frontier vision–language model at one eighth of the latency, and a 4B model matches the frontier model in verifying whether the previous action succeeded. We release a benchmark generator, a break-even formula for when a check pays, and three rules for agent developers: `https://github.com/multimodal-ai-engineering/preflight-bench`.

Keywords Computer-use agents · GUI agents · Vision–language models · Multimodal perception · Benchmarks

1 Introduction

Consider an agent registering a user for a triathlon. It opens the race page, captures a screenshot immediately, and spends four seconds deciding to select *Register* for the sprint distance. During those four seconds, a results banner finishes loading at the top of the page and shifts the content down by one row. The click lands on *Register* for the Olympic distance. The agent's decision was correct for the screenshot it planned on; the page it acted on had changed. Figure 1 shows the same failure on a news feed from our benchmark.

Most computer-use agents follow this pattern: capture a screenshot, plan with a large multimodal model, and click at screen coordinates [1–3]. Planning takes 1–13 s [1, 2]. Browser-testing frameworks guard against this failure by verifying,

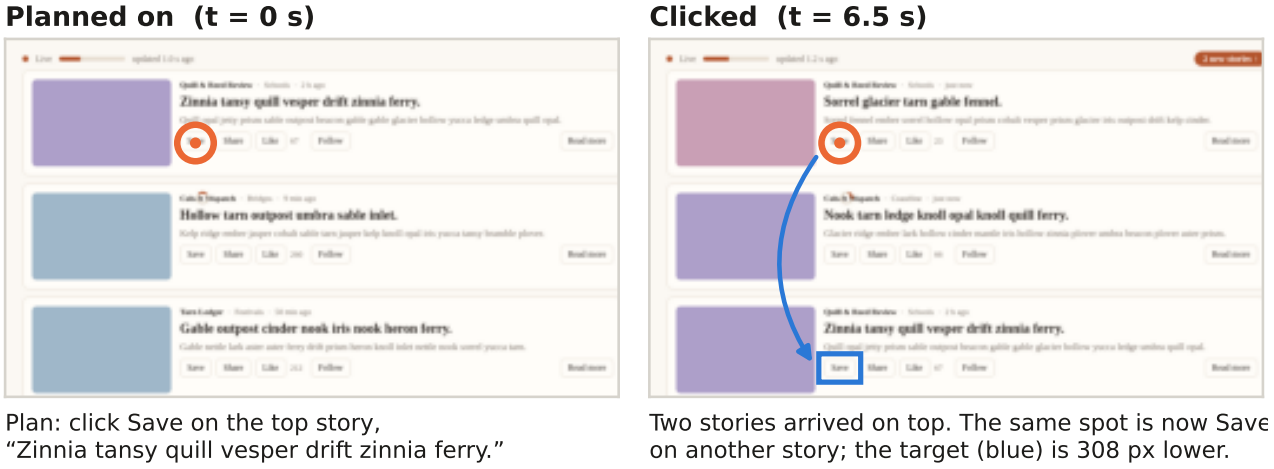


Figure 1: A stale click. The agent plans to save the top story of a news feed (left). While it plans, two new stories arrive at the top, and 6.5 s later its click lands on the Save button of a different story (right). The click succeeds on the wrong item, and the page reports no error. Benchmark page with invented content; ground truth from the page's element tree.

immediately before each click, that the target element is present, stable and able to receive the click [4]. Agents that click at coordinates omit this verification. How often the omission causes errors on real pages, and which form of perception should perform the verification, had not been measured.

The prevailing assumption and our research questions. Existing work treats the planning delay as the source of risk. PUSV measured a 6.5-second interval between screenshot and click and protects it with pixel re-verification [1]; AOI and pre-compiled policy trees aim to shorten the interval [2, 5]. Under this assumption, the natural remedy is a fast visual check immediately before each click, which we call a *preflight*. We tested the assumption and the remedy through three research questions. Throughout, a planned click is *stale* when, at the moment of the click, its target has moved, disappeared or been covered.

RQ1. When does the screenshot become stale? During the planning interval, or at another point?

RQ2. Which form of perception should verify the screen? Pixel comparison, a small vision–language model (VLM), or a frontier model?

RQ3. When is a check worth its cost? Each check adds latency to every step and occasionally raises false alarms.

Findings. *RQ1:* the prevailing assumption does not hold, and planning time has little effect. On fully loaded pages, 0.4% of planned clicks became stale within 30 s. The risk is concentrated in the first second after a page opens, when 17% of planned clicks become stale because the page is still loading. Waiting until network activity stops removes most of this risk, and most widely used agents do not wait long enough. *RQ2:* where a check remains necessary (live dashboards, maps, shared documents and irreversible actions), the form of perception should match the question. Whether the target is still in place is a question about a few hundred pixels at one location; direct pixel comparison answers it as accurately as the best model tested, with fewer false alarms and a fraction of the latency. Whether the previous action succeeded is a question about what the screen now reports, and a 4B model answers it as accurately as the frontier model. *RQ3:* a break-even formula based only on the planner's latency and the check's latency and accuracy predicted when a check pays, agreeing with held-out data on 95% of decisions.

Implications. The findings yield three low-cost rules (Section 6): wait for the page to finish loading before capturing the screenshot; before an irreversible click, verify the target with a pixel check (0.3 s, no model); and when the check fails, re-plan from a new screenshot instead of adjusting the original click. Figure 3 in Section 6 contrasts one agent step under current practice and under these rules.

Contributions. (1) The first measurement of how often an agent's planned click becomes stale on real web pages, locating the risk in page loading, together with an evaluation of the post-navigation wait rules of ten open-source agents. (2) A direct comparison of pixel methods, OCR, small VLMs and frontier models on the three questions a pre-click check must answer. (3) A break-even formula for when a check pays, validated on held-out data. (4) An open benchmark generator that renders moving web pages with exact timing and labels every example from the page's own state. All experiments were preregistered, and we report every outcome, including refuted hypotheses.

2 Three questions, and the checks compared

A preflight runs immediately before the agent clicks and answers one of three questions: *(1) Is the target still where it was planned? (2) Is a dialog or pop-up obstructing it? (3) Did the previous action succeed?* The first two concern a small region of pixels at one location. The third concerns content: whether the screen now reads "Registered" or "Payment failed." This difference is why no single form of perception suits all three.

Pixel check. The check retains two screenshots captured 200 ms apart at planning time. Pixels that differ between them, such as spinners, blinking cursors and tickers, change independently of the agent and are excluded. The check then searches the current screen for the target's planned image patch within 40 pixels of its original position and requires the best match to exceed any lookalike by a clear margin. Two further tests determine whether an element now covers the target and whether a dialog has appeared. The output is *proceed* or *re-plan*. The check takes 312 ms, of which 200 ms is the interval between the two screenshots; no model is involved.

Model checks. We posed the same questions to small open VLMs (Qwen3-VL 2B and 4B, SmolVLM2 2.2B), an OCR model (Florence-2) and two hosted models (Claude Haiku 5.5 and Claude Opus 5.5). Each model received a crop of the planning screenshot around the target, the full current screenshot and the question; for the third question, it received the screenshots before and after the action and a one-sentence description of the expected effect.

PUSV, a published security check. PUSV [1] protects agents from adversaries who alter the screen deliberately. It compares the target region and the full screen with the planning screenshot and halts on any change, which is appropriate

under an adversarial threat model. We reimplemented it from the paper, both as published and tuned to a 2% false-alarm rate, to evaluate how a security check performs on ordinary pages.

3 When is a check worth its cost?

A check adds latency to every step and saves time only when it intercepts a click that would have been stale. Let C denote the agent's planning time, δ the check's latency, r the share of stale clicks it detects (recall), f its false-alarm rate, and k the cost of a stale click measured in planning cycles ($k = 2$ for an ordinary click, covering the wasted plan and the recovery; higher for a payment or a deletion). If p is the probability that the target changes before the click, a check pays when

$$p \;>\; p^{*} = \frac{\delta/C + f}{r(k-1) + f}. \tag{1}$$

Three consequences follow. A faster planner raises the threshold, because a stale click then wastes less time. A riskier action lowers it, because each miss is more costly. A slow check rarely pays, because its own latency extends the interval it is meant to protect. The derivation and two rearrangements, giving the maximum affordable false-alarm rate and check latency, are provided in the repository.

The same reasoning selects between two checks for a single question. A more expensive check B is preferable to a cheaper check A when the additional problems it detects are worth more than its additional latency and false alarms. With q the rate of the problem and M the cost of a missed problem in seconds,

$$qM\,(r_B - r_A) \;>\; (\delta_B - \delta_A) + (1-q)(f_B - f_A)C. \tag{2}$$

4 Experimental setup

Real pages (RQ1). We used 31 public pages approved for read-only access, including clocks, population counters, earthquake and weather maps, reference sites and status pages. On each visit we selected up to 20 visible links and buttons and tracked, for 30 s, whether a click at each element's planned position would still reach it. Tracking began from three states: a fully loaded page, immediately after a scroll, and immediately after the page opened, at the moment the browser has parsed the page's HTML (*DOM-ready*), which usually precedes the arrival of images, fonts and data. Three rounds at different times of day yielded about 1,300 tracked targets per state. A second experiment captured the planning screenshot under seven wait rules after the page opened. A third reimplemented the post-navigation wait rule of ten open-source agents, as written in their current code, and applied it to the same pages; the agents themselves were not executed.

Benchmark (RQ2, RQ3). A check remains necessary on pages that change while the agent plans, such as live dashboards, maps and shared documents, and before irreversible clicks. Real pages change too rarely to measure a check's accuracy, so we built a benchmark of six synthetic pages that move by design: a static form, a market dashboard, a news feed, a shared document edited by a second user, a panning map and a drawing board. Pages render in Chromium with a paused clock, so the interval between planning and clicking is exact (6.5 s, the interval PUSV measured [1]). Within that interval the generator introduces one event: no change, target moved, lookalike moved, target covered, target removed, dialog shown, or slow load. Ground truth comes from the page's element tree at the moment of the click; the checks receive only pixels. Each page has four layouts, two for tuning and two held out for testing. A third tier replays the screens under remote-desktop conditions (latency, compression and scaling). The benchmark contains 4,608 click pairs and 3,600 action-outcome pairs, and all thresholds were frozen before test labels were examined.

Live loop. A Claude Haiku 5.5 planner clicked named elements on the moving test pages for 400 steps per condition, with and without the pixel check.

5 Results

5.1 RQ1: screenshots become stale while the page loads

Figure 2 (left) shows how long a planned click remains valid. On fully loaded pages, 0.4% of planned clicks were stale at every interval from 0.5 to 30 s, and the share did not increase over time. After a scroll, the share was 1.0%. Immediately after a page opened, 14.5% were stale by 0.5 s and 17% by 1 s, after which the share stopped increasing: the page

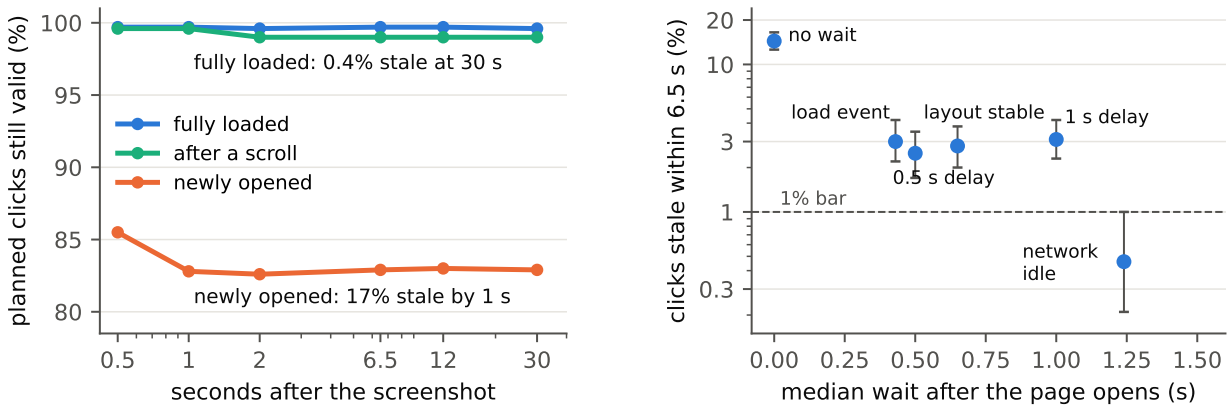


Figure 2: Left: share of planned clicks that remain valid on 31 public pages, by page state. Right: median wait of each rule after the page opens, and the share of clicks it leaves stale (95% CI). Only waiting for network activity to stop reaches 1%.

Table 1: Left: wait rules for the planning screenshot after a page opens, timed from DOM-ready. Right: post-navigation wait rules of ten open-source agents (code reviewed October 2026, default settings), reimplemented in our harness and applied to the same 31 pages; the agents themselves were not executed. Stale: share of planned clicks whose target moved, disappeared or was covered within 6.5 s.

Wait rule	Wait	Stale (95% CI)	Agent (wait rule)	Stale (95% CI)	Wait
no wait	0 s	14.4% (12.6–16.5)	Anthropic CU demo (2 s delay)	2.0% (1.3–2.9)	2.02 s
0.5 s delay	0.50	2.5% (1.7–3.5)	browser-use (load or idle)	4.0% (3.1–5.2)	0.94
load event	0.43	3.0% (2.2–4.2)	OpenAI CUA sample (no wait)	15.4% (13.5–17.4)	0.35
layout stable 0.5 s	0.65	2.8% (2.0–3.8)	Stagehand v4 (no wait)	12.5% (10.8–14.5)	0.34
boxes stable 0.5 s	0.81	1.8% (1.2–2.7)	Skyvern (load, 1 s, animations)	**0.46%** (0.21–1.0)	2.03
1 s delay	1.00	3.1% (2.3–4.2)	Agent S3 (1 s delay)	11.7% (10.1–13.6)	1.02
network idle	1.24	**0.46%** (0.21–1.0)	OpenHands (load or idle)	2.6% (1.9–3.7)	0.93
			Playwright MCP (load)	3.0% (2.2–4.1)	0.88
			UI-TARS-desktop (1 s delay)	9.1% (7.6–10.8)	1.01
			magentic-ui (3 s delay, then load)	1.2% (0.7–1.9)	3.34

completes its layout in a burst that ends within about one second. The difference between newly opened and fully loaded pages was 17 percentage points (95% CI 9.9–24.7), ranging from 5 points on dashboards to 38 points on a reference site.

How long to wait. Table 1 (left) compares wait rules. Only one fell below 1%: waiting until the page has issued no network requests for 500 ms (Playwright's *networkidle*), which left 0.46% of clicks stale at a median wait of 1.24 s (90th percentile 3.2 s). Fixed delays of 0.5 or 1 s, the browser's *load* event, and waiting for layout shifts to stop all left 2.5–3.1%. The load event fires before much of the content arrives, and the browser's layout-shift signal does not register content that fills a fixed-size container or overlays a target, as occurs on maps.

Current agent practice. Table 1 (right) reports the ten agents' wait rules. They leave between 0.46% and 15.4% of planned clicks stale, and eight of ten exceed 1%, as preregistered. The two agents that capture the screenshot as soon as the HTML is parsed leave 12–15%. The two that wait a fixed second leave 9–12%: their delay starts when the server first responds, so 7% of their screenshots are captured before the HTML has been parsed. Only Skyvern, which waits for the load event, then about one second, then for animations to finish, matches the network-idle rule. Some systems wait longer after their own actions, such as AOI's 2 s [2], so this result applies to page navigation.

5.2 RQ2: pixels answer *where*, models answer *what*

Table 2 reports each check's own decisions on the held-out moving pages. For the first question, whether the target is still in place, the pixel check detected 82% of stale clicks at a 4% false-alarm rate in 0.3 s; Opus detected 83% at 12% in 2.4 s. The smaller models failed in opposite directions: Haiku reported a move on almost every pair, including valid ones, while Qwen3-VL 2B and SmolVLM2 reported no move on almost every pair. Qwen3-VL 4B detected 66% at a 37% false-alarm rate. On real pages, PUSV as published flagged 21% of clicks that were still valid, because ordinary pages change continually; the pixel check flagged 0.5%.

For the third question, whether the previous action succeeded, the ranking reverses. Qwen3-VL 4B detected every

Table 2: Decisions of each check on held-out moving pages: share of problems detected (recall), share of valid cases incorrectly flagged (false alarms), and median latency. Small open models ran on balanced subsets; hosted models on every pair.

Question	Check	Detected	False alarms	Latency (ms)
Target still in place? (1,064 stale clicks)	pixel check	**82%**	**4%**	312
	PUSV, tuned to 2% false alarms	24%	0.3%	5
	Qwen3-VL 2B	0.2%	0%	1,450
	Qwen3-VL 4B	66%	37%	2,425
	Claude Haiku 5.5	98%	95%	1,276
	Claude Opus 5.5	83%	12%	2,366
Dialog obstructing? (500 dialogs)	pixel dialog detector	72%	1%	312
	Qwen3-VL 4B	95%	14%	2,425
	Claude Opus 5.5	100%	7%	2,366
Previous action failed? (620 failures)	pixel change score	90%	0%	312
	Florence-2 OCR	89%	0%	4,115
	Qwen3-VL 4B	**100%**	**0%**	2,814
	Claude Opus 5.5	100%	0%	2,366

failure with no false alarms, matching Opus, and the 2B model and Florence-2's OCR performed nearly as well. Dialog detection falls between the two: the pixel detector rarely raises a false alarm but misses about a quarter of dialogs on layouts it was not tuned on, whereas the models detect nearly all of them at false-alarm rates of 7–15%.

This division is consistent with prior findings on VLMs: their image encoders capture fine positional detail that their language output does not report [6], and frontier models miss most small differences between two screenshots [7]. To our knowledge, this is the first direct comparison of pixel methods and VLMs for verifying an exact click position.

5.3 RQ3: when a check pays, and how to respond when it fires

Decisions from the formula. We supplied Equation 1 with values from the tuning layouts only and compared its predictions with a replay of the held-out test set. At the four change rates we measured (fully loaded, scrolled and newly opened pages, and the benchmark), its decision to check or skip matched the replay in 94.7% of 3,000 cases, and its errors favored checking (148 of 158). The exact break-even point was less accurate: within 0.03 of the replay in 70% of cases, below the preregistered 80%. The discrepancies came from slow checks. A check that takes 2.4 s is itself an outdated observation, since the target can move after the check and before the click, a point also made in robotics research on delayed actions [8]. For an ordinary click with a 6.5 s planner, the pixel check pays once more than about 10% of clicks would be stale; Opus does not pay at any rate up to 50%. Equation 2 selected the check that was less costly on the test set in 93% of cases.

Live agent loop. With a Haiku planner on the moving pages, the pixel check reduced stale clicks from 65–69% to 9–11%. Nearly every re-plan it triggered was warranted (581 of 606). Under the preregistered accounting, which charged each re-plan at 6.5 s, it added 28–105% to the time per step and exceeded the preregistered 10% limit. Charged at the planner's measured 1.1 s, it saved 0.2–0.3 s per step.

Responding to a failed check. An apparent shortcut is to locate the target again and adjust the click. This proved unsafe in every configuration. No pixel setting kept incorrect adjustments at or below 1% (best: 11%). Asking Haiku to relocate the element produced an incorrect click 34% of the time and took as long as a full re-plan. In most cases where the check fired, the target had been removed or covered; the pixel method recognized this in 260 of 267 cases, whereas the models clicked an element anyway in up to 216 cases.

6 Three rules for agent developers

1. **Wait for the page to finish loading before capturing the screenshot.** After navigation, wait for network activity to stop, with a cap of about 3 s. This removes most stale clicks at a cost of about one second per page.
2. **Match the check to the question.** On a fully loaded page, skip the check for ordinary clicks; at a 0.4% change rate it does not pay. Before an irreversible action, run the pixel check at the target: it takes 0.3 s, requires no model, and pays once about 1% of such clicks would be stale. After an action that commits a change, verify the outcome with a small model or OCR; a frontier model offers no gain for this question.

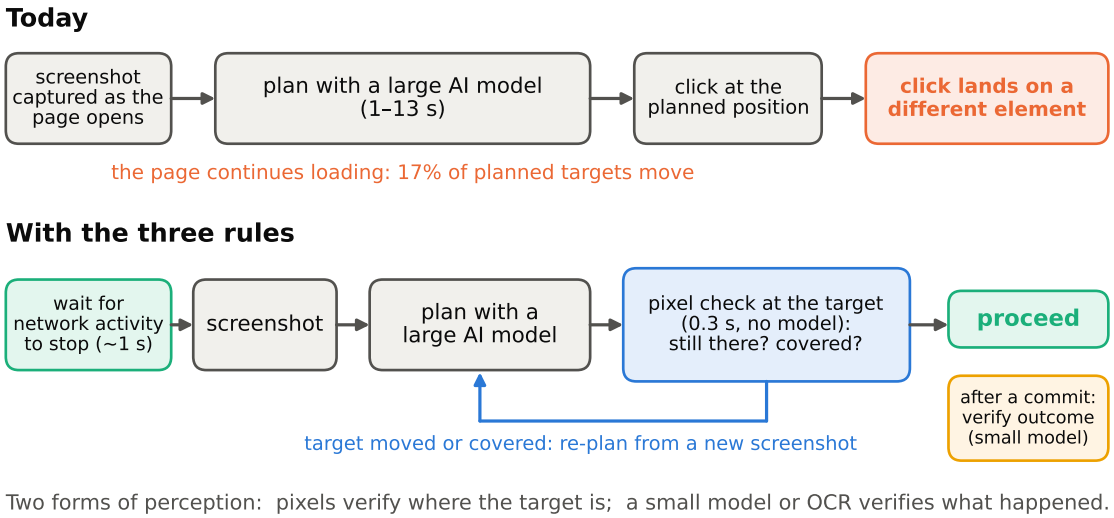


Figure 3: One agent step under current practice (top) and under the three rules (bottom). Most stale clicks arise when the screenshot is captured while the page is still loading. Before the click, a pixel check confirms that the target remains where it was planned; after an action that commits a change, a small model or optical character recognition (OCR) confirms that it succeeded.

3. **When the check fires, re-plan from a new screenshot.** Do not adjust the original click.

The broader implication concerns modality. A frontier multimodal model is well suited to planning and to reading what a screen reports, and poorly suited to confirming that a few hundred pixels remain in place. Equation 2 makes this choice quantitative. The same timing issue likely affects training data: pipelines that record "the frame before the action" by a fixed rule [3] may capture many frames during page loading. Measuring this is a natural next study.

7 Related work

PUSV [1] measured the 6.5 s interval between screenshot and click and protects it against adversaries with pixel re-verification; Atomicity for Agents [9] monitors page changes in browser agents. Both halt on any change and do not measure how often ordinary pages change, which Liu et al. [10] identify as an open question. AOI [2] and pre-compiled policy trees [5] shorten the interval by acting faster. The limitations of VLMs in fine spatial judgment are documented by BlindTest [6], DiffSpot [7] and VisCritic [11], and Belcak et al. [12] argue for small models in agent workflows. Robotics addresses a 40–400 ms form of the same delay by predicting the robot's own motion [8, 13–15]; web pages offer no comparable motion model, which explains why adjusting the click fails here. Browser-testing frameworks wait for elements to stabilize before acting [4].

8 Limitations and preregistration

The benchmark pages change far more often than real pages (about 70% versus 0.4% of planned clicks); they serve as a stress test for checks, while the real pages provide the base rate. The real-page set comprises 31 public pages in one browser over three days; authenticated applications that update continuously may behave differently. The agent results reimplement each agent's wait rule; the agents were not run end to end. The perfect outcome-detection score at 4B suggests that our failure set is relatively easy. We used a single planner, and about 70% of its own errors stemmed from similar labels produced by our task generator, which no pre-click check can detect.

We preregistered eleven hypotheses across four registrations, each frozen before its data was collected. Four were supported: the formula's check-or-skip decisions match held-out data; Equation 2 selects the less costly check; clicks become stale more often immediately after navigation than on a loaded page; and at least eight of ten agents' wait rules leave more than 1% of clicks stale. Seven were refuted, including that a model of 4B parameters or fewer would come within 2 points of Opus on every question, that the live check would add less than 10% to step time, that the break-even point would fall within 0.03 in 80% of cases, that a 0.5 s layout-shift wait would reach 1%, and that model-based click adjustment would be safe. All outcomes are reported in the repository.

9 Conclusion

For computer-use agents, the principal risk lies in page loading more than in planning: screenshots become stale mostly within the first second after a page opens, and a one-second wait removes most of that risk. Where a check remains necessary, pixel comparison should verify where the target is, a small model should verify what happened, and a failed check should trigger a new plan.

Code and data. The benchmark generator, pages, checks, the four preregistrations, raw model outputs and analysis code are available at `https://github.com/multimodal-ai-engineering/preflight-bench`. Real-page screenshots are not released because they contain third-party content.

Disclaimer. This is independent research conducted outside any employment, with no funding and no use of non-public information. It is not endorsed by the author's employer or by any vendor named here.

References

[1] Wenpeng Xu. Temporal UI state inconsistency in desktop GUI agents: Formalizing and defending against TOCTOU attacks on computer-use agents. *arXiv preprint arXiv:2604.18860*, 2026.

[2] Bojie Li and Noah Shi. Agent-computer observation interfaces enable dynamic computer use. *arXiv preprint arXiv:2606.29472*, 2026.

[3] Xinyuan Wang, Bowen Wang, Dunjie Lu, et al. OpenCUA: Open foundations for computer-use agents. *arXiv preprint arXiv:2508.09123*, 2025.

[4] Microsoft. Playwright documentation: Auto-waiting and actionability; Page.waitForLoadState. `https://playwright.dev/docs/actionability`, 2026. Accessed October 2026.

[5] Zihan Dong, Rui Qian, Qishi Zhan, Dongshen Peng, Kaixin Li, and Yu Li. Why are GUI agents correct but late? decode on the decision-time critical path, tested with pre-compiled policy trees. *arXiv preprint arXiv:2607.28399*, 2026.

[6] Pooyan Rahmanzadehgervi, Logan Bolton, Mohammad Reza Taesiri, and Anh Totti Nguyen. Vision language models are blind: Failing to translate detailed visual features into words. *arXiv preprint arXiv:2407.06581 (v6, 2025)*, 2024.

[7] Linhao Zhang, Aiwei Liu, Yuan Liu, and Xiao Zhou. DiffSpot: Can VLMs spot fine-grained visual differences in web interfaces? *arXiv preprint arXiv:2605.29615*, 2026.

[8] Brian Zhu, Momen Khalil, E Harrison, Emanuele Poggi, et al. Learning to act while waiting: RL finetuning of generalist robot policies under inference latency. *arXiv preprint arXiv:2608.23831*, 2026.

[9] Linxi Jiang, Zhijie Liu, Haotian Luo, and Zhiqiang Lin. Atomicity for agents: Exposing, exploiting, and mitigating TOCTOU vulnerabilities in browser-use agents. *arXiv preprint arXiv:2603.00476*, 2026.

[10] Xunzhuo Liu, Bowei He, Xue Liu, Andy Luo, Haichen Zhang, and Huamin Chen. Visual confused deputy: Exploiting and defending perception failures in computer-using agents. *arXiv preprint arXiv:2603.14707*, 2026.

[11] Jiachen Qian. VisCritic: Visual state comparison as process reward for GUI agents. *arXiv preprint arXiv:2606.24525*, 2026.

[12] Peter Belcak, Greg Heinrich, Shizhe Diao, Yonggan Fu, Xin Dong, Saurav Muralidharan, Yingyan Celine Lin, and Pavlo Molchanov. Small language models are the future of agentic AI. *arXiv preprint arXiv:2506.02153*, 2025.

[13] Kevin Black, Manuel Y. Galliker, and Sergey Levine. Real-time execution of action chunking flow policies. *arXiv preprint arXiv:2506.07339*, 2025.

[14] Jiaming Tang, Yufei Sun, Yilong Zhao, Shang Yang, Yujun Lin, Zhuoyang Zhang, James Hou, Yao Lu, Zhijian Liu, and Song Han. VLASH: Real-time VLAs via future-state-aware asynchronous inference. *arXiv preprint arXiv:2512.01031*, 2025.

[15] Kohei Sendai, Maxime Alvarez, Tatsuya Matsushima, Yutaka Matsuo, and Yusuke Iwasawa. Leave no observation behind: Real-time correction for VLA action chunks. *arXiv preprint arXiv:2509.23224*, 2025.